

CIFAKE Image Classification with Explainable AI

Real and AI-Generated Synthetic Image Detection

Tran Quang Trong - 20235565

Abstract

This project builds an end-to-end deep learning pipeline for classifying CIFAKE images into REAL and FAKE classes. Several convolutional neural network architectures are trained and compared, including SimpleCNN, ResNet18, EfficientNet-B0, and DenseNet121. The best model is then analyzed using Grad-CAM to visualize which image regions contribute to the model's prediction. Finally, a Gradio demo is developed to provide an interactive interface for image upload, prediction, and explanation visualization.

Contents

1	Introduction	3
2	Related Work	3
2.1	CIFAKE Dataset	3
2.2	Convolutional Neural Networks	4
2.3	Grad-CAM	4
3	Dataset and Preprocessing	4
3.1	Dataset Structure	4
3.2	Train, Validation, and Test Split	5
3.3	Preprocessing	5
4	Methodology	5
4.1	Training Setup	5
4.2	Evaluation Metrics	6
5	Model Architectures	6
5.1	SimpleCNN	6
5.2	ResNet18	6
5.3	EfficientNet-B0	6
5.4	DenseNet121	6
6	Experimental Results	7
6.1	Model Comparison	7
6.2	Discussion	8

7	Explainability Analysis with Grad-CAM	8
7.1	How Grad-CAM Works	8
7.2	Qualitative Analysis	8
7.3	Quantitative Heatmap Analysis	9
8	Gradio Demo	10
9	Limitations	11
10	Conclusion and Future Work	11

1 Introduction

The rapid development of generative AI has made synthetic images increasingly realistic. This creates a practical need for systems that can distinguish between real images and AI-generated images. However, classification accuracy alone is not enough. It is also important to understand why a model makes a certain prediction.

This project has two main goals. First, it builds deep learning models to classify images from the CIFAKE dataset into REAL and FAKE classes. Second, it applies Grad-CAM to explain model decisions by visualizing important image regions.

The main contributions of this project are:

- Building a complete image classification pipeline for the CIFAKE dataset.
- Comparing multiple CNN-based architectures under the same experimental protocol.
- Selecting the best model based on final test metrics.
- Applying Grad-CAM to analyze model attention.
- Building a Gradio demo for interactive prediction and visualization.

Overall Project Pipeline

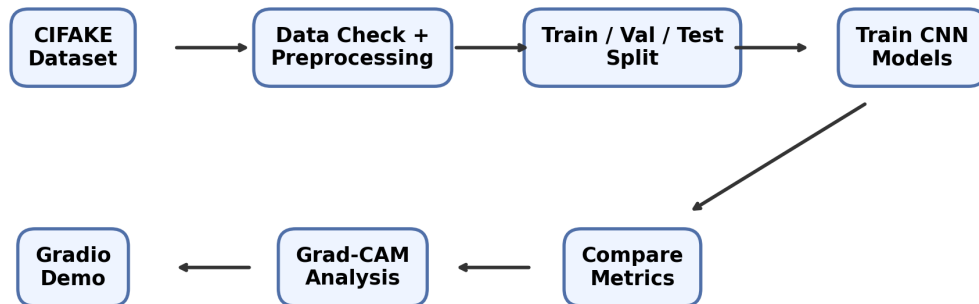


Figure 1: Overall workflow of the CIFAKE image classification and explainability pipeline.

2 Related Work

2.1 CIFAKE Dataset

The CIFAKE dataset contains real images and AI-generated synthetic images. It is designed for binary image classification, where each image belongs to either the REAL class or the FAKE class.

The dataset is available on Kaggle:

<https://www.kaggle.com/datasets/birdy654/cifake-real-and-ai-generated-synthetic-images>

2.2 Convolutional Neural Networks

Convolutional Neural Networks are widely used for image classification tasks. CNNs can learn spatial features from images. Lower layers usually learn simple visual patterns such as edges and textures, while deeper layers learn more complex representations.

2.3 Grad-CAM

Grad-CAM, or Gradient-weighted Class Activation Mapping, is an explainability method for CNN-based models. It uses gradients from the target class score to identify which spatial regions of an image are important for the prediction. The result is a heatmap that can be overlaid on the original image.

3 Dataset and Preprocessing

3.1 Dataset Structure

The dataset is organized as follows:

```
dataset/  
|-- train/  
|   |-- REAL/  
|   |-- FAKE/  
|-- test/  
|   |-- REAL/  
|   |-- FAKE/
```

The dataset statistics are shown in Table 1.

Table 1: CIFAKE dataset statistics		
Split	Class	Number of Images
Train	REAL	50,000
Train	FAKE	50,000
Test	REAL	10,000
Test	FAKE	10,000

All images have a resolution of 32 by 32 pixels.



Figure 2: Example REAL and FAKE images from the CIFAKE dataset.

3.2 Train, Validation, and Test Split

The original training set is split into training and validation subsets:

- Training set: 80 percent of the original training set.
- Validation set: 20 percent of the original training set.
- Test set: the original test set, used only for final evaluation.

This protocol avoids using the test set for early stopping or model selection. The test set is only used after the best validation checkpoint is selected.

3.3 Preprocessing

Each image is loaded using PIL, converted to RGB, and transformed into a PyTorch tensor. The `ToTensor()` transform converts pixel values from the range 0–255 to the range 0.0–1.0.

4 Methodology

4.1 Training Setup

All models are trained from scratch without pretrained weights. The loss function is `BCEWithLogitsLoss`, which is suitable for binary classification. The optimizer is Adam with a learning rate of 0.001.

The training configuration is:

- Maximum epochs: 50
- Early stopping patience: 5
- Batch size: 64
- Validation ratio: 0.2

4.2 Evaluation Metrics

The following metrics are used:

- Loss
- Accuracy
- Precision
- Recall
- F1-score

Accuracy measures the overall percentage of correct predictions. Precision measures how many predicted REAL or FAKE samples are correct. Recall measures how many actual samples of a class are detected. F1-score balances precision and recall.

5 Model Architectures

5.1 SimpleCNN

SimpleCNN is used as the baseline model. It contains convolutional layers, ReLU activation functions, pooling layers, and fully connected layers. Because it is relatively shallow, it is fast to train but may have limited representation capacity.

5.2 ResNet18

ResNet18 uses residual connections. These skip connections allow information to flow directly across layers, making it easier to train deeper networks. This helps the model learn better visual features compared with a simple shallow CNN.

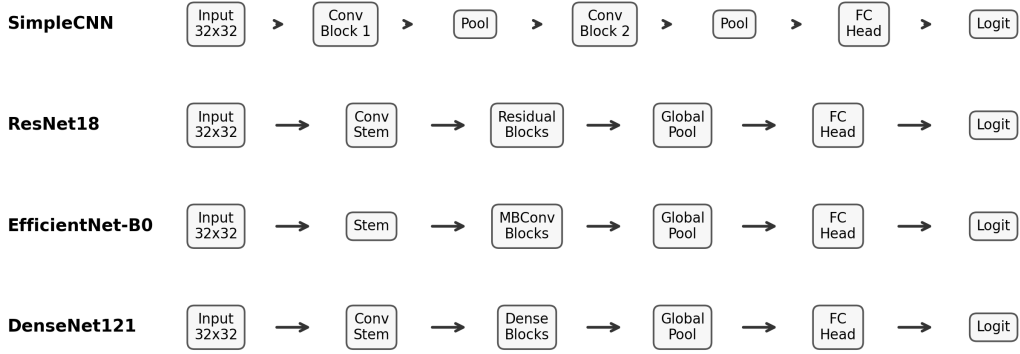
5.3 EfficientNet-B0

EfficientNet-B0 is designed to balance network depth, width, and resolution. It is efficient in terms of parameter usage and often performs well on image classification tasks.

5.4 DenseNet121

DenseNet121 uses dense connections, where each layer receives feature maps from previous layers. This encourages feature reuse and improves gradient flow. In this project, DenseNet121 achieves the best final test performance.

High-Level Model Architecture Comparison



Note: architecture blocks may include convolution, normalization, activation, and feature aggregation operations depending on the model.

Figure 3: High-level comparison of SimpleCNN, ResNet18, EfficientNet-B0, and DenseNet121.

6 Experimental Results

6.1 Model Comparison

Table 2 shows the final test performance of all trained models.

Table 2: Final test metrics of trained models

Model	Test Loss	Accuracy	Precision	Recall	F1-score
SimpleCNN	0.1503	94.265%	95.312%	93.110%	94.198%
ResNet18	0.1371	94.800%	94.190%	95.490%	94.836%
EfficientNet-B0	0.1299	94.955%	95.005%	94.900%	94.952%
DenseNet121	0.1365	95.120%	94.401%	95.930%	95.159%

DenseNet121 obtains the highest test accuracy and F1-score. Therefore, it is selected as the best model for Grad-CAM explainability and the Gradio demo.

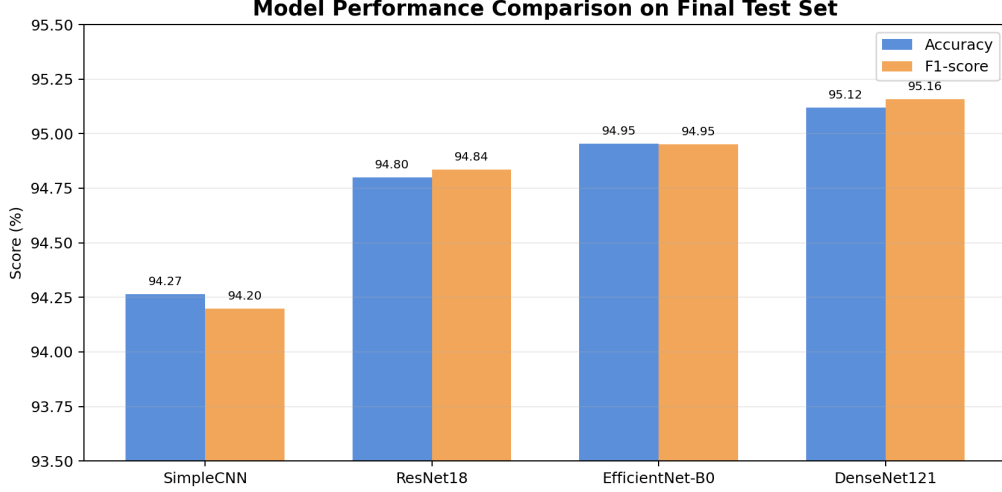


Figure 4: Comparison of model performance on the test set.

6.2 Discussion

SimpleCNN performs well as a baseline but has lower performance than deeper architectures. This is expected because a shallow network may not capture subtle synthetic image artifacts as effectively.

ResNet18 improves over SimpleCNN due to residual connections, which help the model learn deeper feature representations.

EfficientNet-B0 achieves strong results because of its efficient architecture design. However, in this experiment, it does not outperform DenseNet121.

DenseNet121 achieves the best F1-score. One possible reason is that dense connections allow the model to reuse low-level and mid-level visual features, which may be useful for detecting texture and artifact differences between REAL and FAKE images.

7 Explainability Analysis with Grad-CAM

7.1 How Grad-CAM Works

Grad-CAM generates a heatmap by using gradients from the target class score with respect to feature maps in a convolutional layer. The feature maps are weighted by average gradient values, combined, and passed through a ReLU operation. The final heatmap is resized to the input image size and overlaid on the original image.

7.2 Qualitative Analysis

Grad-CAM visualizations are used to compare how different models focus on image regions. The analysis includes REAL images, FAKE images, correct predictions, and incorrect predictions.

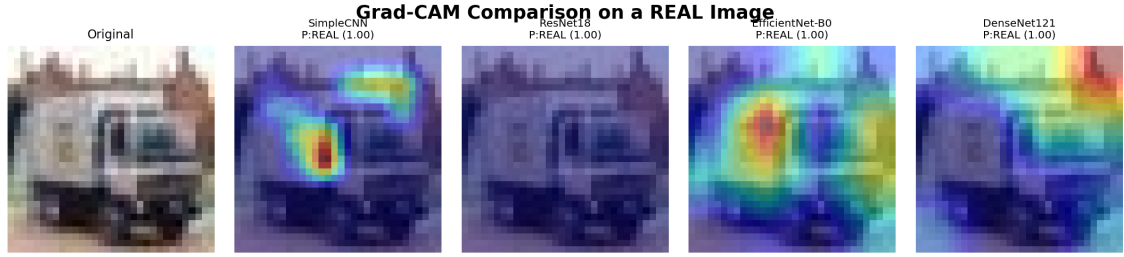


Figure 5: Grad-CAM comparison of different models on a REAL image.

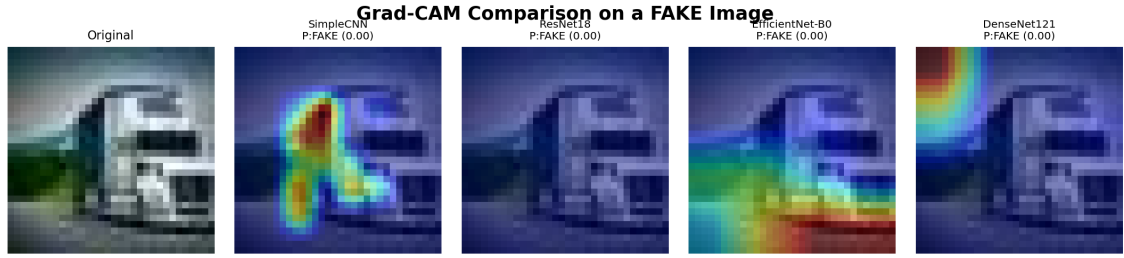


Figure 6: Grad-CAM comparison of different models on a FAKE image.

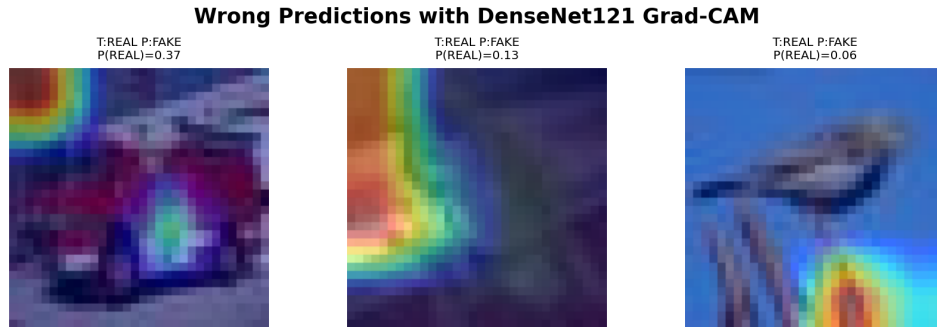


Figure 7: Examples of incorrect predictions and their Grad-CAM explanations.

The visual results suggest that the models may not rely only on semantic object-level information. Instead, they may also focus on texture, background, edges, or artifact-like regions. This is reasonable because the task is not object recognition, but real-versus-synthetic image detection.

7.3 Quantitative Heatmap Analysis

Besides visual inspection, heatmaps are analyzed using the following statistics:

- Coverage: percentage of pixels with high heatmap activation.
- Normalized entropy: how spread out the attention is.
- Peak value: maximum activation strength after normalization.

Table 3: Average normalized heatmap statistics

Model	Coverage	Normalized Entropy	Peak Value
SimpleCNN	0.052	0.701	0.93
ResNet18	0.061	0.534	0.66
EfficientNet-B0	0.277	0.926	1.00
DenseNet121	0.203	0.817	0.94

The heatmap statistics show that better classification performance does not always mean the most spatially concentrated attention. DenseNet121 achieves the best classification result, while its attention can still be broader than ResNet18 in some cases.

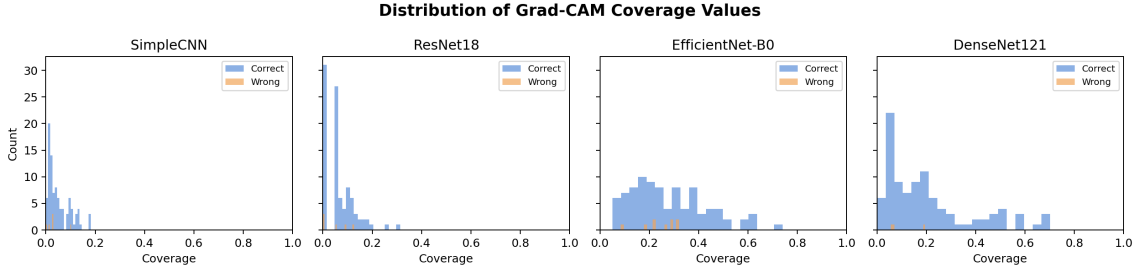


Figure 8: Distribution of Grad-CAM coverage values for each model.

8 Gradio Demo

A Gradio demo is implemented to make the model easier to test interactively. The user can upload an image, run prediction, and view both the classification result and Grad-CAM visualization.

The demo provides:

- Predicted label: REAL or FAKE.
- Probability of REAL and FAKE.
- Grad-CAM heatmap.
- Grad-CAM overlay on the input image.

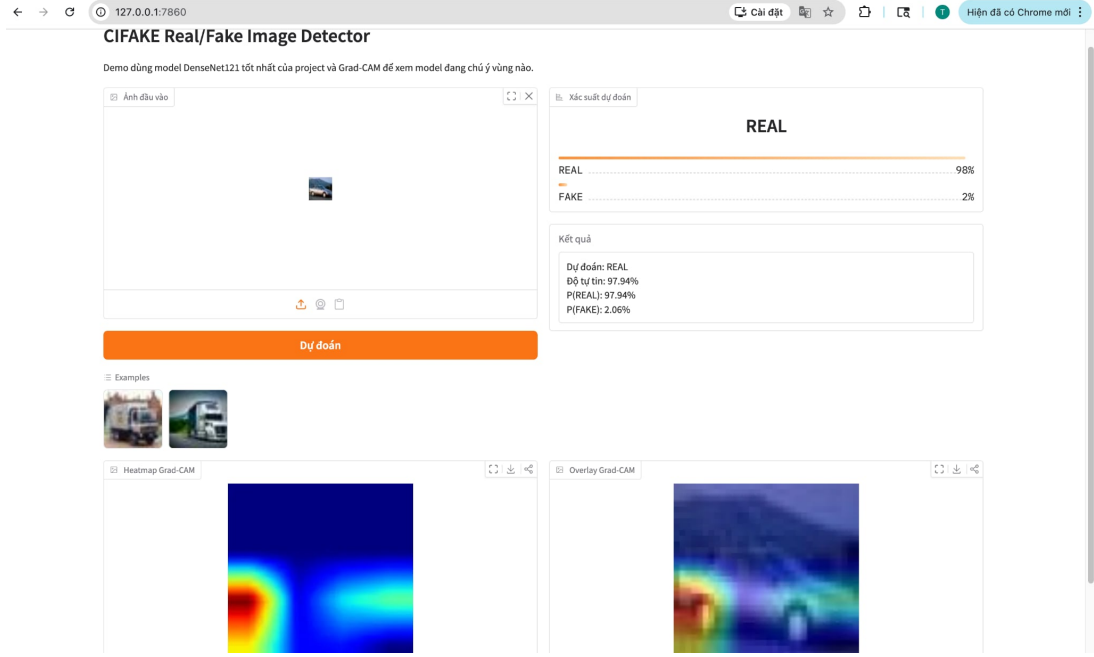


Figure 9: Gradio demo for REAL/FAKE prediction and Grad-CAM visualization.

9 Limitations

This project has several limitations:

- CIFAKE images have low resolution, only 32 by 32 pixels, which makes Grad-CAM visualization coarse.
- The dataset only provides binary labels, so the model cannot analyze different sources of AI-generated images.
- Grad-CAM is a visual explanation method, but it does not prove that the model truly understands semantic content.
- All models are trained from scratch, which may require more training time than fine-tuning pretrained models.

10 Conclusion and Future Work

This project successfully builds a complete pipeline for REAL/FAKE image classification using the CIFAKE dataset. Multiple CNN architectures are trained and evaluated under a strict train-validation-test protocol. DenseNet121 achieves the best final test performance with 95.120% accuracy and 95.159% F1-score.

Grad-CAM is used to visualize model attention and provide insight into how models make predictions. The analysis suggests that models may rely on texture and artifact-level cues, not only semantic object-level features.

Future work may include:

- Testing additional explainability methods such as LIME and SHAP.

- Training on higher-resolution datasets.
- Analyzing AI-generated images by generator type.
- Trying Vision Transformer or pretrained fine-tuning.
- Deploying the Gradio demo online.

References

- Bird, J. J., and Lotfi, A. CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images, 2023.
- Selvaraju, R. R., et al. Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization, ICCV 2017.